

quantum-group: Exact symbolic verification of R-matrix and Yang–Baxter identities for $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$

Taha Berk Terekli*

Department of Mathematics, Yıldız Technical University, Istanbul, Türkiye

* terekli@tahaberk.com, ORCID: [0009-0004-8266-1116](https://orcid.org/0009-0004-8266-1116)

Abstract

Explicit R-matrices of quantum groups are routinely transcribed from the literature, yet whether they satisfy an identity depends on basis order, tensor placement, the coproduct and, for superalgebras, the Koszul sign rule. `quantum-group` is a Python package that builds such matrices exactly with SymPy and returns every identity as an exact residual matrix, whose nonzero entries locate the convention that disagrees. It covers $U_q(\mathfrak{sl}_2)$ modules, tensor products and the fundamental R-matrix, and the graded Yang–Baxter equation for $GL_q(2|1)$. Applied to the thesis it was written for, it exposed an R-matrix that satisfies the Yang–Baxter equation but intertwines the opposite coproduct.

Copyright attribution to authors.

This work is a submission to SciPost Physics Codebases.

License information to appear upon publication.

Publication information to appear upon publication.

Received Date

Accepted Date

Published Date

1

2

Contents

1	Introduction	2
2	Design and workings	3
3	Installation and usage	6
4	Worked examples	7
5	Testing and benchmarks	10
6	Scope, limitations and extensions	11
7	Conclusion	12
8	References	13

12

13

1 Introduction

Quantum groups are one-parameter deformations of classical symmetry algebras. Their representations yield solutions of the quantum Yang–Baxter equation (QYBE), which is why they appear in exactly solvable models of statistical mechanics, in knot invariants and in representation theory [1–4]. Much of the working knowledge in this area is recorded as explicit matrices whose entries are rational functions of the deformation parameter q : an R-matrix on $V \otimes V$, the generators of a module, a change of basis to irreducible summands. Before such a matrix can be reused it has to be checked, and checking it means multiplying 9×9 , 27×27 or 81×81 matrices whose meaning depends on conventions that texts state in different ways, or not at all.

The characteristic mistakes are not arithmetic ones. A matrix is transcribed in a different basis order from the one its source used. R_{13} is formed by conjugating with the wrong permutation. An R-matrix is paired with the opposite of the coproduct it actually intertwines; this still satisfies the QYBE, so the most common test does not detect it. In a $\mathbb{Z}_2$ -graded setting an ordinary flip is used where the Koszul sign rule requires a super-permutation. And a check that is run at a floating-point value of q can hide cancellations or produce spurious nonzero entries. Each of these errors produces output that looks plausible.

quantum-group [5] addresses this with a deliberately narrow design. Every object is an ordinary exact sympy.Matrix [6]; conventions (basis order, coproduct, parities) are fixed in documented functions; and every identity is exposed as a *residual*, the exact matrix LHS–RHS, which is then tested entry by entry. For $U_q(\mathfrak{sl}_2)$ the package constructs the irreducible modules V_n , checks the defining relations and generator-level Hopf identities, builds tensor products through the coproduct, computes Clebsch–Gordan highest-weight vectors, and verifies the fundamental R-matrix: QYBE, braid and Hecke relations and coproduct intertwining. For the quantum supergroup $GL_q(2|1)$ introduced by Çelik and Çelik [7], whose third basis vector is odd, it constructs the published 9×9 R-matrix and verifies the graded Yang–Baxter equation on three factors and on every triple of factors in $V^{\otimes 4}$.

The software was developed for and used in the author’s undergraduate thesis [8], where it produced the thesis’s tables, matrix figures and Yang–Baxter verifications. After the thesis was submitted in June 2026, the software was used to audit it: an explicit intertwining residual showed that the thesis’s R-matrix intertwines the opposite of the coproduct used in the text, an error that passing QYBE, braid and Hecke checks had not revealed (Sec. 4.2). The correction is recorded in the repository (thesis/ERRATA.md, AUDIT.md), and the check is now part of the API and the test suite. No new mathematical results are claimed; the contribution is the software and its verification method.

The intended users are researchers who need to confirm a matrix identity from the literature in their own conventions: mathematical physicists working with R-matrices and integrable models, representation theorists and computer-algebra users who want an inspectable reference calculation, and students reproducing standard quantum-group computations.

Related software. GAP’s QuaGroup package [9–11] and SageMath [12], which provides a QuantumGroup interface to it, work with the quantized enveloping algebra $U_q(\mathfrak{g})$ of any semisimple Lie algebra $\mathfrak{g}$: they construct PBW-type and canonical bases, highest-weight modules and their R-matrices; SageMath also implements crystals for the quantum superalgebras $U_q(\mathfrak{gl}(m|n))$ [13]. These systems are far broader, and for general $U_q(\mathfrak{g})$ work they are the appropriate tools. quantum-group does not replace them. Its added value is that of a small verification layer for concrete matrix calculations: it keeps the basis order and coproduct of a particular text, returns the matrices and residuals themselves rather than only a verdict, runs in an ordinary Python environment, encodes the graded sign rule once for every tensor

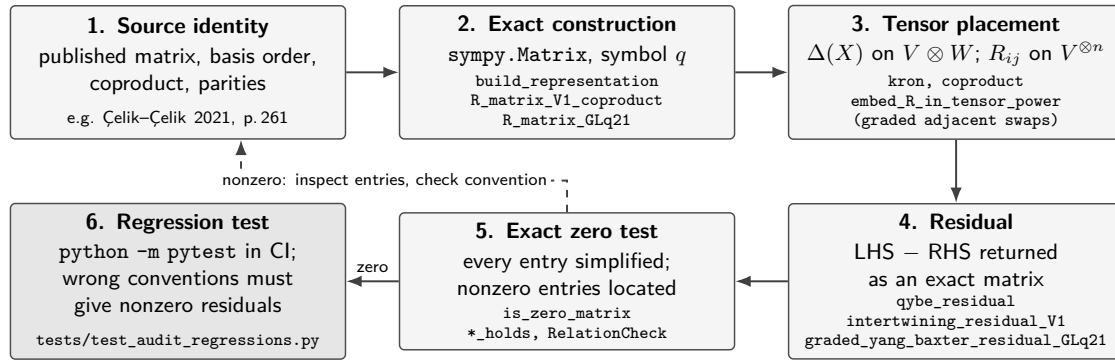


Figure 1: Verification workflow in quantum-group. A published identity and its conventions (1) are turned into exact SymPy matrices (2) and placed on tensor products through the coproduct or through graded adjacent swaps (3). The difference of the two sides is returned as an exact residual matrix (4) and tested entry by entry (5); a nonzero residual points to the entries, and hence the convention, that disagree. Zero residuals, and nonzero residuals for deliberately wrong conventions, are kept as regression tests (6). Function names are those of the package; `kron` is imported from `quantum_group.linalg`, all others from the package namespace.

62 placement, and includes a graded $GL_q(2|1)$ example that lies outside QuaGroup’s documented
63 $U_q(\mathfrak{g})$ scope.

64 **Outline.** Section 2 describes how the package works and why it is built this way. Section 3
65 explains installation and the public interface, Sec. 4 gives worked examples, including the
66 thesis audit, and Sec. 5 documents testing and performance. Section 6 states the scope and
67 the ways the code can be reused or extended.

68 2 Design and workings

69 quantum-group is a pure-Python package of fifteen modules (about 2,100 lines, including
70 docstrings) with no compiled extensions. Figure 1 shows the verification workflow that the
71 modules implement; the design decisions behind each step are described below.

72 2.1 Explicit matrices rather than an abstract algebra engine

73 The package contains no noncommutative rewriting system; the symbols in `generators.py`
74 only display the relations. Every check runs on concrete representation matrices. This keeps
75 each object directly comparable, entry by entry, with a printed matrix, and makes every in-
76 termediate result an ordinary SymPy object that users can print, substitute into or pass to
77 other SymPy code. The price is that a verification holds for the representations on which it is
78 run, and that the cost grows with the dimension of the tensor product (Sec. 5.3). Identities
79 that need a genuinely noncommutative graded algebra, such as the Gauss decomposition of
80 $GL_q(2|1)$, are omitted rather than approximated with commuting symbols.

81 2.2 Residual-first verification

82 Each check follows one pattern: build the left-hand side, build the right-hand side, return their
83 difference, and decide whether every entry is zero. Functions named `*_residual` (`qybe_`

84 `residual`, `braid_relation_residual`, `intertwining_residual_V1`, `graded_yang_`
 85 `baxter_residual_GLq21`, and others) return the exact difference matrix; the correspond-
 86 ing `*_holds` functions apply `is_zero_matrix` to it; and `verify_on_representation`
 87 returns, for each defining relation, a `RelationCheck` record holding both the Boolean and
 88 the residual. Returning the residual rather than only a Boolean is what makes a failed check
 89 informative: the positions and values of the nonzero entries identify which basis vectors and
 90 which convention disagree (Sec. 4).

91 The zero test simplifies each entry with `sympy.simplify`. A `True` result means that every
 92 entry was reduced to the exact value 0; a `False` result is not by itself a proof of inequality,
 93 because `simplify` is not a decision procedure. For the rational functions of q produced by the
 94 package, a nonzero entry can be confirmed with `sympy.cancel`, which the regression tests
 95 do.

96 2.3 Conventions for $U_q(\mathfrak{sl}_2)$

97 For a highest weight n , the function `build_representation(n)` returns the $(n + 1)$ -
 98 dimensional module V_n as a `Representation` record with explicit matrices E , F , K and K^{-1}
 99 in the basis $v_0, \dots, v_n$, where K is diagonal, F is a lower shift and E carries the products of
 100 q -integers. The coproduct is fixed as

$$\Delta(E) = E \otimes 1 + K \otimes E, \quad \Delta(F) = F \otimes K^{-1} + 1 \otimes F, \quad \Delta(K) = K \otimes K, \quad (1)$$

101 and `coproduct` and `tensor_product` realize it with Kronecker products in the lexicographic
 102 basis order (left factor slow, right factor fast). `find_highest_weight_vectors` solves for
 103 the vectors annihilated by E in each weight space of $V_m \otimes V_n$, giving one highest-weight vector
 104 per Clebsch–Gordan summand.

105 **Two R-matrix conventions.** On $V_1 \otimes V_1$, in the basis $v_0 \otimes v_0, v_0 \otimes v_1, v_1 \otimes v_0, v_1 \otimes v_1$, the
 106 package provides

$$R = \begin{pmatrix} q & 0 & 0 & 0 \\ 0 & 1 & q - q^{-1} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & q \end{pmatrix}, \quad R_{21} = \begin{pmatrix} q & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & q - q^{-1} & 1 & 0 \\ 0 & 0 & 0 & q \end{pmatrix}, \quad (2)$$

107 where $R_{21} = PRP$ and P is the flip of the two factors. `R_matrix_V1` returns the historical
 108 upper-triangular matrix R , the image, rescaled by $q^{1/2}$, of Drinfeld’s universal R-matrix [1]
 109 for the coproduct opposite to Eq. (1); accordingly it satisfies $R \Delta^{\text{op}}(X) = \Delta(X)R$, i.e. it inter-
 110 twines the *opposite* of the coproduct (1). Rather than change its output, which would silently
 111 alter results already computed with it, version 1.1.0 keeps it unchanged and adds `R_matrix_`
 112 `V1_coproduct` (R_{21}), `R_check_V1_coproduct` (PR_{21}) and `intertwining_residual_`
 113 `V1`, which satisfy $R_{21} \Delta(X) = \Delta^{\text{op}}(X)R_{21}$. Both conventions satisfy the QYBE, and both braided
 114 forms satisfy the braid relation and the Hecke relation $\check{R} - \check{R}^{-1} = (q - q^{-1})\mathbb{1}$, with eigenval-
 115 ues q (multiplicity three) and $-q^{-1}$; they differ in which tensor-product submodules are the
 116 braid eigenspaces. Keeping both, with the convention stated in each docstring, lets earlier
 117 calculations be reproduced exactly while new work uses the compatible matrices. The same
 118 policy applies to names: a renamed function keeps its old name as a thin wrapper that emits
 119 a `DeprecationWarning`.

2.4 Graded tensor embeddings for $GL_q(2|1)$

For $GL_q(2|1)$ the basis parities $(0, 0, 1)$ are an explicit argument, and the graded flip is built once, by `super_permutation_matrix`, as

$$P_s(e_i \otimes e_j) = (-1)^{p(i)p(j)} e_j \otimes e_i. \quad (3)$$

`embed_R_in_tensor_power` places a $d^2 \times d^2$ operator on factors (i, j) of $V^{\otimes n}$ by first embedding it on the adjacent pair $(i, i+1)$ with Kronecker products and then conjugating by graded adjacent swaps $S_k = \mathbb{1}^{\otimes k} \otimes P_s \otimes \mathbb{1}^{\otimes (n-k-2)}$ until the second factor reaches position j . The Koszul sign rule is therefore encoded in a single matrix and reused for every placement, instead of being written out separately for R_{13} , R_{14} , R_{24} and so on. The construction is correct only for even (parity-preserving) operators, and the function checks this before embedding. Because the parity list is an argument, the same function also embeds operators on other super vector spaces, and an all-even parity list reproduces the ungraded placement. `R_matrix_GLq21` returns the 9×9 R-matrix exactly as printed on page 261 of Ref. [7], and `graded_yang_baxter_residual_GLq21` forms $R_{12}R_{13}R_{23} - R_{23}R_{13}R_{12}$ on $V^{\otimes 3}$.

2.5 Exact parameters

The default parameter is a nonzero SymPy symbol, and the identities are rational in q , holding wherever their denominators are defined. Integers and exact rationals passed as q are converted to exact SymPy numbers, so `build_representation(2, 2)` yields entries such as $5/2$ rather than 2.5 ; only floating-point values supplied by the user are approximate. q -integers and q -binomials are evaluated as Laurent polynomials, not as quotients, so that specializations with removable singularities, such as $q = -1$ or roots of unity, give the correct values instead of $0/0$.

2.6 Validation and failure modes

Public functions reject invalid input with specific errors rather than returning a misleading matrix: non-integral or negative highest weights, sizes and indices raise `TypeError` or `ValueError`; `verify_on_representation` rejects non-square or mismatched generator matrices and the singular values $q \in \{0, 1, -1\}$ of the defining commutator quotient; `qybe_residual` rejects matrices that are not $d^2 \times d^2$; and `embed_R_in_tensor_power` rejects invalid positions, parity lists and odd operators. Symbolic input is not restricted: any SymPy expression can be used as q .

2.7 Package structure

The modules form three layers. A small foundation provides the default symbol q (declared nonzero), q -integers, q -factorials and q -binomials (`utils`); the Kronecker product `kron/kron_list` and the zero-residual test `is_zero_matrix` (`linalg`); and internal argument checks (`_validation`). Construction modules build the matrices: $U_q(\mathfrak{sl}_2)$ modules (`representations`), coproduct, counit and antipode (`hopf`), tensor products and Clebsch–Gordan vectors (`tensor`), $U_q(\mathfrak{sl}_2)$ R-matrices (`r_matrix`) and the $GL_q(2|1)$ R-matrix, super-permutation and embeddings (`supergroup_gl21`). Verification functions sit next to the constructions they check and all use the same foundation. Auxiliary modules provide classical and root-of-unity specializations (`limits`), the crystal graph $B(n)$ as a `NetworkX` [14] graph (`crystal`), `Matplotlib` [15] weight and crystal plots (`visualization`) and a convenience facade, `QuantumGroupSL2`. The main functions are re-exported from the package namespace and listed in `__all__`.

3 Installation and usage

3.1 Requirements, installation and availability

quantum-group requires CPython 3.10 or later and runs on any operating system; it has no compiled code and needs no special hardware. It is tested in continuous integration on Linux, macOS and Windows (Sec. 5) and locally on Ubuntu 24.04 (x86-64). The runtime dependencies, installed automatically, are SymPy ≥ 1.10 [6], NetworkX ≥ 2.6 [14] and Matplotlib ≥ 3.5 [15]; continuous integration exercises the oldest supported versions (SymPy 1.10.1, NetworkX 2.6.3, Matplotlib 3.5.3). pytest [16] is needed only to run the test suite (extra [test]).

The version described here is 1.1.0, identified by commit 103d5384e70e8b0b746b296a0bdbd2d156ba8dde of the code repository and tagged as the GitHub Release v1.1.0. It is installed from the Python Package Index, or directly from that commit:

```
python -m pip install quantum-group==1.1.0
python -m pip install \
  "git+https://github.com/TerekliTahaBerk/quantum-groups@v1.1.0"
python -c "import quantum_group; print(quantum_group.__version__)" # 1.1.0
```

The tag v1.1.0 points to commit 103d5384e70e8b0b746b296a0bdbd2d156ba8dde; the full hash can be used in its place. To run the tests and benchmarks, clone the repository and install it with the test extra:

```
git clone https://github.com/TerekliTahaBerk/quantum-groups.git
cd quantum-groups && python -m pip install -e ".[test]"
python -m pytest # 202 tests
python benchmarks/benchmark.py --repeats 3 --max-n 5 # Table 3
```

Code availability. Repository: <https://github.com/TerekliTahaBerk/quantum-groups> (MIT licence). Release: <https://github.com/TerekliTahaBerk/quantum-groups/releases/tag/v1.1.0>. Package: <https://pypi.org/project/quantum-group/1.1.0/>. Archive: Zenodo, doi:10.5281/zenodo.23015576, a source archive of the same commit [5]. Listing 1 is examples/sample_verification.py of the release. Listings 2–4 were written for this paper after version 1.1.0 was released; they use only the public interface of 1.1.0 and are in scipost/examples/ on the repository’s main branch, each with its expected output.

3.2 Overview of the public interface

Table 1 lists the main functions by task. Functions that construct q -dependent objects accept an optional parameter `q_sym` (default: the package symbol `q`); all return ordinary SymPy objects or plain Python records of them, and each states its convention in the docstring.

3.3 Documentation and support

Documentation consists of the README (installation, quickstart, conventions, limitations), module and function docstrings that state the conventions used (basis order, coproduct, parities) and the relation each check tests, CONTRIBUTING.md, CHANGELOG.md, an exploratory Jupyter notebook, and MANUSCRIPT_CODE_MAPPING.md, which links the computational claims of the thesis to their implementation and tests. Questions, bug reports and feature requests are handled through the GitHub issue tracker (<https://github.com/TerekliTahaBerk/quantum-groups/issues>). CONTRIBUTING.md describes the development set-up, the validation command and the conventions for new checks (a `*_residual` function with a `*_holds` companion, input validation with a test for each error path, and a stated reference for any convention). The package is maintained by

Table 1: Main public functions of quantum-group 1.1.0, grouped by task. All are importable from the package namespace.

Task	Functions
$U_q(\mathfrak{sl}_2)$ modules and relations	build_representation, verify_on_representation, all_relations_hold, symbolic_relations
Hopf structure	coproduct, counit, antipode, verify_coassociativity, verify_antipode, verify_all_hopf_axioms
Tensor products	tensor_product, find_highest_weight_vectors, cg_summands, cg_decomposition_summary, swap_matrix, kron_list
$U_q(\mathfrak{sl}_2)$ R-matrices	R_matrix_V1, R_matrix_V1_coproduct, R_check_V1, R_check_V1_coproduct, R_check_eigenvalues
Generic residuals	qybe_residual, qybe_holds, braid_relation_residual, braid_relation_holds, hecke_skein_relation_check, intertwining_residual_V1, intertwining_holds_V1, is_zero_matrix
$GL_q(2 1)$ and grading	R_matrix_GLq21, super_parity_gl21, super_permutation_matrix, embed_R_in_tensor_power, all_Rij_GLq21, graded_yang_baxter_residual_GLq21, graded_yang_baxter_holds_GLq21, local_ybe_on_four_tensor_GLq21, braid_far_commutativity_residual_GLq21
Specializations	q_integer, q_factorial, q_binomial, classical_limit, root_of_unity_substitution
Crystals and plots	build_crystal, e_tilde, f_tilde, plot_weight_diagram, plot_crystal_graph

its author on a best-effort basis, with no institutional support or guaranteed response time; changes are recorded in CHANGELOG.md and versions follow semantic versioning.

4 Worked examples

The outputs below were produced with quantum-group 1.1.0, SymPy 1.14 and Python 3.11; each is stored next to its script and is compared automatically with a fresh run.

4.1 A first verification session

Listing 1 (examples/sample_verification.py) checks an installation in a few seconds. It verifies the defining relations on V_2 , the QYBE and coproduct intertwining of R_{21} , and the graded Yang–Baxter equation of $GL_q(2|1)$; it then repeats the last check with the $GL_q(2|1)$ R-matrix placed by ordinary, all-even swaps, as a negative control.

```

from quantum_group import (
    build_representation, verify_on_representation, R_matrix_V1_coproduct,
    qybe_holds, intertwining_holds_V1, R_matrix_GLq21,
    embed_R_in_tensor_power, graded_yang_baxter_residual_GLq21,
    is_zero_matrix)

rep = build_representation(2)          # V_2 of U_q(sl_2): 3x3 matrices
print("E on V_2:", rep.E.tolist())
checks = verify_on_representation(rep.E, rep.F, rep.K, rep.K_inv)
print("relations:", {k: c.holds for k, c in checks.items()})

R = R_matrix_V1_coproduct()           # R_21, matches the package coproduct

```

```

229 print("QYBE:", qybe_holds(R))
230 print("intertwines Delta:",
231       [intertwining_holds_V1(X) for X in ("E", "F", "K", "K_inv")])
232
233 Y = graded_yang_baxter_residual_GLq21() # GL_q(2|1), parities (0, 0, 1)
234 print("graded YBE residual:", Y.shape, "zero:", is_zero_matrix(Y))
235
236 # Negative control: the same R placed with ordinary (all-even) swaps.
237 R12, R13, R23 = (embed_R_in_tensor_power(R_matrix_GLq21(), ij, 3, [0, 0, 0])
238                 for ij in [(0, 1), (0, 2), (1, 2)])
239 bad = (R12 * R13 * R23 - R23 * R13 * R12).expand()
240 print("ungraded residual, nonzero entries:",
241       {(i, j): x.factor()
242        for (i, j), x in sorted(bad.todok().items()) if x != 0})

```

Listing 1: First verification session: examples/sample_verification.py.

```

244 Output (examples/sample_verification_expected.txt; the last line is wrapped here):
245 E on V_2: [[0, q + 1/q, 0], [0, 0, q + 1/q], [0, 0, 0]]
246 relations: {'R1': True, 'R2': True, 'R3': True, 'R4': True}
247 QYBE: True
248 intertwines Delta: [True, True, True, True]
249 graded YBE residual: (27, 27) zero: True
250 ungraded residual, nonzero entries: {(8, 24): 2*q**2*(q - 1)**2*(q + 1)**2,
251   (17, 25): 2*q**2*(q - 1)**2*(q + 1)**2}

```

252 The negative control shows why the grading matters: with ordinary swaps, exactly two entries
253 of the 27×27 residual are nonzero, both equal to $2q^2(q-1)^2(q+1)^2$. Rows and columns are
254 indexed lexicographically, $e_a \otimes e_b \otimes e_c \mapsto 9a + 3b + c$, so entry (8,24) couples $e_0 \otimes e_2 \otimes e_2$
255 with $e_2 \otimes e_2 \otimes e_0$; both involve the odd vector e_2 twice, which is exactly where the Koszul sign
256 $(-1)^{p(i)p(j)}$ differs from +1.

257 4.2 Application: which coproduct does an R-matrix intertwine?

258 This is the check that exposed the convention error in the author's thesis [8]. The thesis
259 used the coproduct (1) together with the upper-triangular matrix R of Eq. (2). R satisfies
260 the QYBE and its braided form satisfies the braid and Hecke relations, so every Yang–Baxter-
261 type test passed. The relation that fails is the one that ties an R-matrix to a coproduct,
262 $R\Delta(X) = \Delta^{\text{op}}(X)R$. Listing 2 forms this residual for both matrices of Eq. (2) with public
263 functions only, so that the same code can be pointed at any other 4×4 R-matrix.

```

264 import sympy as sp
265 from quantum_group import (build_representation, coproduct, swap_matrix,
266                           R_matrix_V1, R_matrix_V1_coproduct, qybe_holds,
267                           is_zero_matrix)
268
269 D = coproduct(build_representation(1)) # Delta(X) on V_1 (x) V_1, 4x4
270 P = swap_matrix(2) # ordinary flip of the two factors
271
272 def intertwining_residual(R, X):
273     """R Delta(X) - Delta^op(X) R, with Delta^op(X) = P Delta(X) P."""
274     return R * D[X] - P * D[X] * P * R
275
276 for name, R in [("R_matrix_V1", R_matrix_V1()),
277                ("R_matrix_V1_coproduct", R_matrix_V1_coproduct())]:
278     print(name, "- QYBE:", qybe_holds(R))
279     for X in ("E", "F", "K"):
280         res = intertwining_residual(R, X)
281         nonzero = {ij: sp.factor(x) for ij, x in sorted(res.todok().items())
282                   if sp.simplify(x) != 0}

```

```
284 print(f" {X}:", "zero" if is_zero_matrix(res) else nonzero)
```

Listing 2: Testing an R-matrix against the coproduct:
scipost/examples/coproduct_check.py.

286 Output (scipost/examples/coproduct_check_expected.txt; long lines wrapped
287 here):

```
288 R_matrix_V1 - QYBE: True
289   E: {(0, 1): (q - 1)*(q + 1), (0, 2): -(q - 1)*(q + 1)/q,
290      (1, 3): (q - 1)*(q + 1)/q**2, (2, 3): -(q - 1)*(q + 1)/q}
291   F: {(1, 0): (q - 1)*(q + 1)/q**2, (2, 0): -(q - 1)*(q + 1)/q,
292      (3, 1): (q - 1)*(q + 1), (3, 2): -(q - 1)*(q + 1)/q}
293   K: zero
294 R_matrix_V1_coproduct - QYBE: True
295   E: zero
296   F: zero
297   K: zero
```

298 Both matrices pass the QYBE, but only R_{21} intertwines Δ . The residual of R is informative in
299 three ways. It vanishes for K , as it must for any weight-preserving matrix, so the problem is
300 not the diagonal part. Every nonzero entry carries the factor $(q - 1)(q + 1)$, so all of them
301 vanish at $q = \pm 1$, where $q - q^{-1} = 0$ and R and R_{21} coincide; the discrepancy is a genuine
302 q -deformation effect. And every nonzero entry couples one of the middle basis vectors $v_0 \otimes v_1$,
303 $v_1 \otimes v_0$ (indices 1 and 2), the block in which R and R_{21} differ, to $v_0 \otimes v_0$ or $v_1 \otimes v_1$. The same
304 residual with Δ and Δ^{op} exchanged vanishes for R , confirming that R belongs to the opposite
305 coproduct. The package now exposes this check as `intertwining_residual_V1`, and the
306 test suite keeps the failing case as a negative control.

307 4.3 Locating a transcription error

308 A matrix typed from a paper can be passed to `qybe_residual` directly. Listing 3 introduces
309 a single sign error in R of Eq. (2), writing $q + q^{-1}$ for $q - q^{-1}$.

```
310 import sympy as sp
311 from quantum_group import q, qybe_residual, is_zero_matrix
312
313 # R-matrix typed from a source with one mistyped sign: q + 1/q at (1, 2),
314 # where the source (the package's R_matrix_V1) has q - 1/q.
315 R_typed = sp.Matrix([[q, 0, 0, 0],
316                     [0, 1, q + 1/q, 0],
317                     [0, 0, 1, 0],
318                     [0, 0, 0, q]])
319
320 Y = qybe_residual(R_typed) # exact 8x8 residual
321 print("QYBE holds:", is_zero_matrix(Y))
322 print("nonzero entries:", {ij: sp.factor(x) for ij, x in sorted(Y.todok().items())
323                        if sp.simplify(x) != 0})
```

Listing 3: A mistyped entry: scipost/examples/transcription_check.py.

325 Output (scipost/examples/transcription_check_expected.txt):

```
326 QYBE holds: False
327 nonzero entries: {(1, 4): -2*(q**2 + 1)/q, (3, 6): 2*(q**2 + 1)/q}
```

328 Only two of the 64 entries are nonzero. In the basis $e_a \otimes e_b \otimes e_c \mapsto 4a + 2b + c$ of $V_1^{\otimes 3}$, they
329 couple $v_0 \otimes v_0 \otimes v_1$ with $v_1 \otimes v_0 \otimes v_0$ and $v_0 \otimes v_1 \otimes v_1$ with $v_1 \otimes v_1 \otimes v_0$, so the search for the error
330 can start from the entries that move a v_1 between the first and third factors. A floating-point
331 check at a sample value of q would report a failure as well, but not the exact form of the error.

332 4.4 Clebsch–Gordan decomposition

333 Listing 4 decomposes $V_2 \otimes V_1 \cong V_3 \oplus V_1$ by computing one highest-weight vector per summand
 334 in the tensor-product basis $v_i \otimes w_j \mapsto 2i + j$.

```

335 from quantum_group import (build_representation, tensor_product,
                             find_highest_weight_vectors, cg_summands)

T = tensor_product(build_representation(2), build_representation(1))
print("summands V_k, k =", cg_summands(2, 1))
for k, v in find_highest_weight_vectors(T):
    print(f"k = {k}:", list(v))

```

Listing 4: Highest-weight vectors of $V_2 \otimes V_1$: `scipost/examples/clebsch_gordan.py`.

336 Output (`scipost/examples/clebsch_gordan_expected.txt`):

```

337 summands V_k, k = [1, 3]
338 k = 3: [1, 0, 0, 0, 0, 0]
339 k = 1: [0, (-q**2 - 1)/q**3, 1, 0, 0, 0]

```

340 The highest-weight vector of the V_1 summand is $v_1 \otimes w_0 - q^{-3}(q^2 + 1)v_0 \otimes w_1$; its q -dependent
 341 coefficient is fixed by the coproduct (1): with Δ^{op} it would be $-(q^2 + 1)$ instead of $-q^{-3}(q^2 + 1)$,
 342 which is why the coproduct formulas are stated in the documentation of the `hopf` and `tensor`
 343 modules and in the README.

344 5 Testing and benchmarks

345 5.1 Test suite and continuous integration

346 Every change is checked by one command, `python -m pytest`, which runs the test suite
 347 with `pytest` [16] together with the package `doctests`. At version 1.1.0 the suite comprises 198
 348 test cases in ten files and four `doctests` (202 in total, all passing). GitHub Actions runs it
 349 on every push and pull request on Python 3.10, 3.11, 3.12 and 3.13; once with the oldest
 350 supported dependency versions (SymPy 1.10.1, NetworkX 2.6.3, Matplotlib 3.5.3 on Python
 351 3.10); and, after building and installing a wheel, runs the README quickstart and the example
 352 of Listing 1 from outside the source tree, comparing the latter’s output with the stored copy.
 353 A further job repeats the wheel installation, the examples and the full test suite on Linux,
 354 macOS and Windows runners (Python 3.12). Listing 1 gives the same output with SymPy
 355 1.14 on Python 3.12 and with SymPy 1.10.1 on Python 3.10. Table 2 summarizes what the
 356 tests establish. Besides algebraic identities, the suite benchmarks the implementation against
 357 published reference values: every nonzero entry of the $GL_q(2|1)$ R-matrix is compared with
 358 the printed source [7], and the normalization of the $U_q(\mathfrak{sl}_2)$ R-matrix is recomputed from
 359 Drinfeld’s universal R-matrix [1].

360 5.2 Negative controls and independent re-derivations

361 A suite that only checks that correct identities hold cannot show that the checks are able
 362 to fail. Several tests therefore apply a known wrong convention (the opposite coproduct, the
 363 ordinary instead of the graded flip, a mutated counit) and require the residual to be nonzero, in
 364 some cases at specified entries with specified values: the two nonzero entries of the ungraded
 365 residual in Listing 1 are fixed by a test, and the failure of the historical R-matrix to intertwine
 366 the coproduct (Listing 2) is kept as a test requiring a nonzero residual. These tests fix the
 367 scientific content of the conventions, not only the behaviour of the code. Two further tests are

Table 2: Evidence provided by the test suite. Entries in the last column abbreviate `tests/test_name.py`. “Negative control” marks tests that apply a known wrong convention and require a nonzero residual.

Evidence	What is checked	Test file
Exact entries	Every nonzero entry of the 9×9 $GL_q(2 1)$ R-matrix against the display on printed page 261 of Ref. [7]; exact entries of the 4×4 $U_q(\mathfrak{sl}_2)$ R-matrix; shift and weight structure of V_n	<code>audit_regressions,</code> <code>r_matrix,</code> <code>representations</code>
Coproduct convention	$R_{21}\Delta(X) = \Delta^{\text{op}}(X)R_{21}$ for $X = E, F, K, K^{-1}$; negative control: the historical matrix fails for E and F	<code>audit_regressions</code>
Clebsch–Gordan bases	Complete change of basis for $V_1 \otimes V_1, V_2 \otimes V_2, V_3 \otimes V_2$: invertible over $\mathbb{Q}(q)$ and intertwining all four generators; braid eigenvalues q and $-q^{-1}$ on the V_2 and V_0 summands	<code>audit_regressions,</code> <code>tensor</code>
Graded embeddings	All six placements R_{ij} on $V^{\otimes 4}$ equal an independently written signed basis-action formula that builds no swap matrix; disjoint placements commute	<code>audit_regressions,</code> <code>supergroup_gl21</code>
Grading	Graded YBE residual (27×27) is zero; negative control: with the ordinary flip exactly two entries are nonzero, with their values fixed	<code>supergroup_gl21,</code> <code>audit_regressions</code>
Exact parameters	No floating-point atoms for integer, rational, $q = -1$ and $q = i$ inputs; Laurent-polynomial q -integers and q -binomials agree with generic formulas after substitution; root-of-unity orders validated	<code>audit_regressions,</code> <code>limits</code>
Algebraic identities	Defining relations on V_n ; coassociativity, counit and antipode on generators; QYBE, braid and Hecke relations	<code>relations, hopf,</code> <code>r_matrix</code>
Input validation	Every validation path raises the documented error; shared helpers agree with <code>sympy.kronecker_product</code>	<code>validation_and_</code> <code>linalg</code>

independent re-derivations: the four-factor placements are compared with a formula that acts on basis vectors directly, and the normalization of the fundamental R-matrix is recomputed from Drinfeld’s universal R-matrix [1].

5.3 Performance benchmarks

Operators on $V^{\otimes n}$ are stored as dense symbolic matrices, $3^n \times 3^n$ for $GL_q(2|1)$, which keeps every entry inspectable but grows exponentially. Table 3 reproduces the recorded benchmark (`benchmarks/results.md`), generated by the command given in Sec. 3.1. Tensor powers $n \geq 6$ have not been benchmarked. A sparse or tensor-network backend could reach larger n , at the cost of the entry-by-entry inspectability that motivates the design; the calculations the package targets involve at most four or five factors.

6 Scope, limitations and extensions

Scope and limitations. A zero residual establishes an identity on the particular finite-dimensional representation used; it is not a proof in the abstract Hopf algebra, and the package is not a theorem prover. It does not implement general quantum groups. For $GL_q(2|1)$ it implements the R-matrix, the graded Yang–Baxter equation and tensor placements, not the RTT relations, the Hopf superalgebra structure or the Gauss decomposition

Table 3: Wall-clock times (mean and minimum of three runs after a first, separately reported run) from benchmarks/results.md; the smallest case is dominated by timing noise (standard deviation 0.080 s). Environment: Python 3.11, SymPy 1.14, one core of a 2.1 GHz Intel Xeon processor, 27 September 2026.

Computation	n	Matrix size	Mean (s)	Min (s)
QYBE of R for $U_q(\mathfrak{sl}_2)$ (qybe_holds)	3	8×8	0.075	0.028
Graded YBE residual for $GL_q(2 1)$	3	27×27	0.025	0.024
Graded YBE, residual and zero test	3	27×27	0.060	0.055
All placements R_{ij} (all_Rij_GLq21)	4	81×81	0.977	0.954
Local YBE on every triple (local_ybe_on_four_tensor_GLq21)	4	81×81	1.556	1.540
All placements R_{ij} (all_Rij_GLq21)	5	243×243	14.616	14.276
Graded YBE on one triple, incl. embedding	5	243×243	15.221	14.799

of Ref. [7]. It verifies the Hecke relation but does not compute a Markov trace or the Jones polynomial. It works with constant R-matrices; spectral-parameter-dependent solutions of the Yang–Baxter equation are not built in. Its root-of-unity helpers substitute special values into the generic construction; they do not implement the small quantum group. The crystal graph $B(n)$ is a combinatorial model rather than a crystal-basis algorithm.

Reuse. The extension points are ordinary Python functions that accept and return SymPy matrices, and the package is small enough to read in full. Any $d^2 \times d^2$ matrix can be passed to qybe_residual or braid_relation_residual (Sec. 4.3); the intertwining test of Listing 2 works for any R-matrix, module or coproduct formula; and verify_on_representation accepts any four square matrices, so alternative bases or normalizations of V_n can be checked directly. super_permutation_matrix and embed_R_in_tensor_power take an arbitrary parity list and can place any even operator on factors (i, j) of $V^{\otimes n}$ for other $\mathbb{Z}_2$ -graded spaces, such as $\mathbb{C}^{m|n}$ with a different ordering of even and odd basis vectors. Because each check returns an exact matrix, an identity from a paper or thesis becomes a pytest test in a few lines, with a known wrong convention kept alongside it as a negative control; tests/test_audit_regressions.py contains worked examples of this pattern. Outputs can be exported with sympy.latex or compared against matrices computed elsewhere, for example in SageMath, by forming a residual.

Where reuse requires new work. Moving from $\mathfrak{sl}_2$ to general semisimple Lie algebras would need root data, PBW bases and higher-rank modules, which QuaGroup and SageMath already provide. Implementing the RTT relations or the Hopf superalgebra of $GL_q(2|1)$ would need a noncommutative graded algebra rather than matrices. The dense matrix representation limits practical calculations to about five tensor factors of a three-dimensional space. These directions are possible but would be substantial new software rather than reuse of the existing layer.

7 Conclusion

quantum-group turns convention-sensitive identities of quantum-group theory into exact residual matrices that can be inspected entry by entry and kept as regression tests. Its value lies less in the range of algebras it covers, which is deliberately small, than in making the conventions explicit: basis order, coproduct and grading are fixed in documented functions,

the Koszul sign rule is encoded once for every tensor placement, and every check can fail informatively. The thesis audit of Sec. 4.2 shows that this matters in practice: an R-matrix that passes every Yang–Baxter-type test can still belong to the wrong coproduct, and only a residual tied to the coproduct reveals it. Natural next steps are further explicit rank-one and graded R-matrices, which reuse the existing residual and embedding layer unchanged, and a sparse backend for larger tensor powers.

Acknowledgements

The author thanks Prof. Dr. Salih Çelik for supervising the undergraduate thesis from which this software originated.

Author contributions T.B.T. is the sole author: T.B.T. designed and wrote the original implementation; decided on, reviewed and integrated all later changes to the software, including AI-assisted ones; and prepared and approved the manuscript.

Funding information This work received no specific funding.

Competing interests The author has no competing interests to declare.

Use of generative AI No generative AI tools were used during the original development and thesis work (April–June 2026). In September 2026, Claude Code (Anthropic) was used for packaging and continuous-integration configuration, the benchmark script, the shared matrix helpers, input validation and their tests, the sample-verification example, documentation, translating Turkish docstrings and comments into English, a mathematical and technical audit that led to the parameter-handling corrections, the coproduct-compatible R-matrix functions and the audit regression tests, and drafting of this manuscript. The model was Claude Opus 5.5, except for part of the manuscript and submission-material preparation, which used Claude Sonnet 5; commits produced with Claude Code are authored or co-authored by “Claude” in the Git history, with the model named in the commit trailer. No other AI tool was used. Every AI-assisted code change is exercised by the test suite, and every AI-assisted mathematical claim was checked by an executable symbolic test or against the primary source, in particular the published Çelik–Çelik article. The author reviewed and validated all AI-assisted output, made the scientific and design decisions, and takes responsibility for the software and this paper.

References

- [1] V. G. Drinfeld, *Quantum groups*, In *Proceedings of the International Congress of Mathematicians, Berkeley, California, 1986*, vol. 1, pp. 798–820. American Mathematical Society, Providence, RI, <https://www.mathunion.org/fileadmin/ICM/Proceedings/ICM1986.1/ICM1986.1.ocr.pdf> (1987).
- [2] M. Jimbo, *A q -difference analogue of $U(\mathfrak{g})$ and the Yang–Baxter equation*, Lett. Math. Phys. **10**, 63 (1985), doi:[10.1007/BF00704588](https://doi.org/10.1007/BF00704588).
- [3] M. Jimbo, *A q -analogue of $U(\mathfrak{gl}(N+1))$, Hecke algebra, and the Yang–Baxter equation*, Lett. Math. Phys. **11**, 247 (1986), doi:[10.1007/BF00400222](https://doi.org/10.1007/BF00400222).

- [4] C. Kassel, *Quantum Groups*, vol. 155 of *Graduate Texts in Mathematics*, Springer, New York, doi:[10.1007/978-1-4612-0783-2](https://doi.org/10.1007/978-1-4612-0783-2) (1995).
- [5] T. B. Terekli, *quantum-group: Exact SymPy verification of explicit $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$ matrix identities*, Zenodo, doi:[10.5281/zenodo.23015576](https://doi.org/10.5281/zenodo.23015576), Version 1.1.0 (2026).
- [6] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig *et al.*, *SymPy: symbolic computing in Python*, PeerJ Comput. Sci. **3**, e103 (2017), doi:[10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103).
- [7] S. Çelik and S. A. Çelik, *A new quantum supergroup and its Gauss decomposition*, Rep. Math. Phys. **88**, 259 (2021), doi:[10.1016/S0034-4877\(21\)00073-2](https://doi.org/10.1016/S0034-4877(21)00073-2).
- [8] T. B. Terekli, *Quantum grup yapılarının Python ortamında modellenmesi [Modelling quantum group structures in Python]*, Undergraduate thesis, Department of Mathematics, Yıldız Technical University, Istanbul, <https://github.com/TerekliTahaBerk/quantum-groups/tree/main/thesis> (2026).
- [9] The GAP Group, *GAP – Groups, Algorithms, and Programming*, Version 4.16.1, <https://www.gap-system.org> (2026).
- [10] W. A. de Graaf, *QuaGroup: computations with quantum groups (GAP package)*, Version 1.8.4, <https://github.com/gap-packages/quagroup> (2024).
- [11] W. A. de Graaf, *Computing with quantized enveloping algebras: PBW-type bases, highest-weight modules and R-matrices*, J. Symb. Comput. **32**, 475 (2001), doi:[10.1006/jsc.2001.0479](https://doi.org/10.1006/jsc.2001.0479).
- [12] The Sage Developers, *SageMath, the Sage Mathematics Software System*, doi:[10.5281/zenodo.8042260](https://doi.org/10.5281/zenodo.8042260), Version 10.9, <https://www.sagemath.org> (2026).
- [13] G. Benkart, S.-J. Kang and M. Kashiwara, *Crystal bases for the quantum superalgebra $U_q(\mathfrak{gl}(m, n))$* , J. Amer. Math. Soc. **13**, 295 (2000), doi:[10.1090/S0894-0347-00-00321-0](https://doi.org/10.1090/S0894-0347-00-00321-0).
- [14] A. A. Hagberg, D. A. Schult and P. J. Swart, *Exploring network structure, dynamics, and function using NetworkX*, In G. Varoquaux, T. Vaught and J. Millman, eds., *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pp. 11–15. Pasadena, CA, https://conference.scipy.org/proceedings/SciPy2008/paper_2/ (2008).
- [15] J. D. Hunter, *Matplotlib: a 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007), doi:[10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [16] H. Krekel, B. Oliveira, R. Pfannschmidt, F. Bruynooghe, B. Laughner, F. Bruhin *et al.*, *pytest: simple powerful testing with Python*, <https://github.com/pytest-dev/pytest> (2004).